

# Deep Dive: Volatility & the Four Hiding Places

The upstream case's method, demonstrated together — the two new challenges stay yours.

**Stefanus Eko Prasetyo**

Lab-7 · companion to the Lab 07 lecture

# What we cover today

## 1 The four hiding places

clipboard, search history, environment, command line - the case's map

## 2 Plugin workflows

the exact commands per hiding place, with reading guides

## 3 Correlation drill

one fictional incident walked end to end

## 4 Strings in two encodings

the safety net under every plugin

## 5 Your turn

Challenge B (the triage export) and Challenge C (the strings blob)

# The clipboard (illustrative output)

```
root@dfir:~/lab07#  
$ python2 vol.py -f leader.mem --profile Win7SP1x64 clipboard  
Legend: [S] = Stable, [V] = Volatile  
Offset      Handle  Type      Data  
-----  
0xffffffff900.. 0x201  CF_TEXT  flag{d3m0_cl1pb0ard}  
  
# users copy passwords, notes, and secrets all day;  
# whatever was LAST on the clipboard is still in RAM
```

# The search history

```
root@dfir:~/lab07#  
  
# browser URLs from memory:  
$ vol.py ... iehistory | grep -i search  
.../query?q=how+to+contact+the+courier  
  
# and the raw sweep (any browser):  
$ strings -el leader.mem | grep 'query?q='  
.../query?q=flag%7B...%7D  
# URL-encoded braces - decode once (Lab 04!)
```

## Reading tips

- ▶ search queries reveal intent
- ▶ iehistory covers IE cache
- ▶ other browsers: strings sweep
- ▶ queries may be URL-encoded

# The environment variable (illustrative output)

```
root@dfir:~/lab07#  
$ vol.py ... envvars | grep -iE 'flag|token|secret|key'  
Pid    Process      Variable      Value  
----    -  
1816   taskhost.exe  COR_ENABLE_   1  
2904   notepad.exe   MYFLAG        flag{d3m0_3nv_v4r}  
  
# legitimate vars are boring - filter for the interesting  
# names, then check WHICH process owns the block
```

# The executed command (illustrative output)

```
root@dfir:~/lab07#  
$ vol.py ... consoles # full history: prompt + output  
C:\Users\leader> echo staging phase complete  
C:\Users\leader> echo final token: flag{d3m0_c0ns013}  
  
$ vol.py ... cmdscan # just the commands, all consoles  
2023-04-02 14:58:12 > run_hidden.bat  
  
# cmdscan = what was typed; consoles = typed + shown
```

# One incident, four plugins, one story

| Step | Finding                                                   | Plugin   |
|------|-----------------------------------------------------------|----------|
| 1    | ivkui.exe PID 3472 - odd name, late start, under explorer | pstree   |
| 2    | lives in AppData\Roaming - user-space persistence         | cmdline  |
| 3    | spawns powershell -enc <base64> - decode it               | cmdline  |
| 4    | ESTABLISHED to 45.9.148.23:4444 from PID 3596             | netscan  |
| 5    | console history names the handover token                  | consoles |

*No single plugin tells the story - the story is the join across them. That join is what your Challenge B writeup must show.*

# The upstream case - your command card

```
root@dfir:~/lab07#  
  
# 0. download the dump (link in the lab README)  
$ python2 vol.py -f leader.mem imageinfo  
# 1-4. one command per hiding place:  
$ vol.py ... clipboard  
$ vol.py ... envvars | grep -iE 'flag|token'  
$ vol.py ... consoles # and cmdscan  
$ vol.py ... iehistory | grep -i search  
# + strings -el for the raw safety net
```

## Honesty note

- ▶ the dump lives on Google Drive
- ▶ your instructor verifies the 4 flag values with these exact commands
- ▶ the method above IS the graded answer
- ▶ citations: plugin + output line

# Reading a triage export - the generic recipe

```
root@dfir:~/lab07#  
  
# 1. pslist: scan names, PPIDs, start times  
# 2. pstree: find the odd child under a normal parent  
# 3. cmdline: where does it live? what did it spawn?  
#    -enc? base64 -d and read it  
# 4. netscan: match PIDs to foreign endpoints  
# 5. consoles: the human typed something
```

## Export caveats

- ▶ exports can be stale or partial
- ▶ note the acquisition timestamp
- ▶ corroborate across ALL sections
- ▶ grep is your section navigator

# Strings hunting - the generic recipe

```
root@dfir:~/lab07#  
$ strings mem_strings.bin | less  
$ strings -el mem_strings.bin | less  
# sweep BOTH; classify what you find:  
# URLs, env-looking VAR=..., command lines,  
# base64 blobs, and 'almost-flags'  
# decode each layer before judging it
```

## Mind the tricks

- ▶ UTF-16 hides from plain strings
- ▶ ROT13 looks like noise until it isn't
- ▶ base64 blobs decode to words
- ▶ one artifact is the real one

# Common mistakes, proven habits

| Mistake                                | Habit instead                                    |
|----------------------------------------|--------------------------------------------------|
| Running plugins without imageinfo      | profile first - wrong map, wrong world           |
| Reading pslist names only              | correlate PPID, start time, cmdline, and netscan |
| Skipping -el on strings                | Windows text is UTF-16LE - sweep both encodings  |
| Stopping at the first flag-like string | decode every layer; decoys are planted           |
| Pasting flags without citations        | plugin + PID + line, or it didn't happen         |

## CASE CLOSED - THE FULL TIMELINE

### Lab 04

web logs: 192.168.0.106 breaches the app via traversal, RCE, SQLi

### Lab 05

the wire: batman, a reverse shell on 4444, root, a 5555 backdoor

### Lab 06

the disk: the USB stick, its ghost entry, forged timestamps

### Lab 07

memory: the live machine - processes, sockets, and secrets at rest in RAM

*One intrusion, four evidence layers - that is digital forensics.*

# Two artifacts. Everything you've learned.

01

## Challenge B — 4.2

triage\_output.txt: a Volatility export from an imaging-fault case. Find the process that does not belong, decode what its child downloaded, name the C2 endpoints, and read what the user typed at the end.

02

## Challenge C — 4.3

mem\_strings.bin: a raw blob with four artifact types inside - but only one true flag, and it will not show itself to plain ASCII strings. Use every decoder from the series.

03

## Working paper

pre-lab first; staged hints in section 4; cite plugin + PID (or strings mode) for every claim; hash the artifacts.